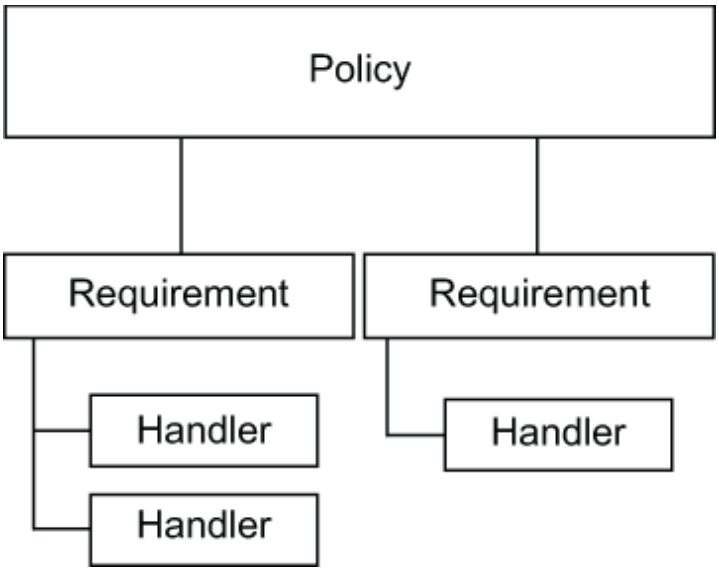


Figure 10.6 Policies consist of one or more requirements, each having one or more handlers.

Using this pattern, it is possible to build complex authorization policies that enable fine-grained control over who can reach which parts of an application. In addition to protecting endpoints, it is also possible to apply authorization policies within a Razor page itself, so, for example, you can toggle the visibility of parts of the UI, depending on the current user's claims.

When you used the `AuthorizationPolicyBuilder.RequireRole` method earlier, a requirement of type `RolesAuthorizationRequirement` was created that specifies that the nominated role is required. Other methods are available on the `AuthorizationPolicyBuilder` that enable you to express common policies simply by using other built-in requirements and handlers (table 10.2).

Table 10.2 Common methods for building simple policies